

Intermediate (timeouts and retry-with-backoff) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real retry loop avoids making an outage worse). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

This has two separate parts. First, prove to yourself that a timeout actually works — point a call at an address that will never answer, and give it a short maximum wait time. Watch it fail on purpose, quickly, instead of hanging.

Second, build a loop that tries a call multiple times, waiting longer between each failed attempt, and gives up after a fixed number of tries instead of forever.

Things to use:

- `requests.get(url, timeout=(3, 5))` — 3 seconds to connect, 5 seconds to read.
- `except requests.exceptions.Timeout:` — catches specifically a timeout, not any other failure.
- `for attempt in range(5):` — try up to 5 times.
- `time.sleep(2 ** attempt)` — wait longer each time before retrying.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exercise has two independent skills that later combine into one function: a **timeout**, and **retry with exponential backoff**.

For the timeout: `requests.get(url, timeout=(connect_timeout, read_timeout))` — pass a tuple so the connect phase and the read phase each get their own limit. Point it at an address that will never respond (a non-routable IP like `http://10.255.255.1` is reliable for this — it just sits there). It should raise `requests.exceptions.Timeout` once the limit is hit, not hang.

For the retry loop: `for attempt in range(max_attempts):` around a `try/except`, and on failure, `time.sleep(2 ** attempt)` before looping again. `2 ** attempt` is exponential backoff — it doubles the wait each time (1s, 2s, 4s, 8s...).

The exact pieces:

- `timeout=(connect, read)` — a 2-tuple, not a single number, lets you set the connect phase and the read phase separately.
- `requests.exceptions.Timeout` — catching this instead of the broader `RequestException` means you only retry *this* kind of failure here.
- **Breaking out of the loop on success** — the loop needs a way to stop early once a call actually succeeds.
- **What happens after the loop ends with no success** — raise your own clear error (or re-raise the last exception) rather than letting the function silently return `None`.

Sketch the loop's structure — what's inside the `try`, what happens in the `except`, when does it give up — before checking Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Two problems with the plain `2 ** attempt` backoff only show up once you imagine more than one caller. First: if a server goes down and 50 different clients all started their first request within the same second, plain exponential backoff makes every one of them retry at *exactly* the same moments — 1s later, 2s later, 4s later — so they keep colliding with each other in a synchronized wave instead of spreading out. **Jitter** — adding a small random amount on top of the backoff formula — breaks that synchronization, since no two clients wait the exact same total time even if they started at the exact same moment.

Second: think about what a caller of your retry function actually needs to test. A test that calls the real function and waits through real `2 ** attempt` seconds of sleeping is slow and flaky (it depends on network timing). The fix is a design change, not a code trick: pull the backoff *math* into its own tiny function, separate from the loop that actually sleeps and makes calls — `compute_backoff_delay(attempt) -> float`. That function is pure math (no network, no `time.sleep`), so it can be tested directly and instantly: does attempt 3 really return roughly 8 seconds? You don't need a fake server to answer that.

The extra pieces:

- `random.uniform(0, 1)` — adds a small random extra wait (jitter) on top of the exponential formula.
- `compute_backoff_delay(attempt: int) -> float` as its own function, called by the retry loop but tested on its own.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both build one loop that retries a single caller correctly. Advanced asks what happens once you imagine *many* callers failing at the same moment (jitter, so they don't retry in lockstep) and asks how you'd actually verify the backoff math is right without a slow, flaky test that waits through real sleeps (pulling it into its own pure function). Neither of those problems exists until you stop thinking about "does this one retry loop work" and start thinking about "would this survive being used by a real, busy program."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
part 1 - prove the timeout works:
    try to call a URL that never answers, with a short timeout
    catch the timeout error
    print that it happened

part 2 - retry loop:
    try up to 5 times:
        try the call
        if it works: stop, use the result
        if it fails: wait longer each time, then try again
    if all 5 tries failed: give up, print a clear message
```

Here is almost the whole retry loop — just try running it and reading it line by line:

```
# retry_backoff_practice.py – Intermediate section
import time
import requests

for attempt in range(5):
    try:
        response = requests.get("http://10.255.255.1", timeout=(1, 2))
        break
    except requests.exceptions.Timeout:
        print("attempt", attempt, "timed out")
        time.sleep(2 ** attempt)
```

Expected output if all 5 attempts fail: 5 lines like `attempt 0 timed out` through `attempt 4 timed out`, each followed by a wait, then nothing — right now the loop just falls through silently once `range(5)` runs out. What's missing: what happens if all 5 attempts fail, and the separate small script proving a timeout fires on its own. Add those yourself, then check the Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
part 1:
    try:
        requests.get("http://10.255.255.1", timeout=(1, 2))
    except requests.exceptions.Timeout:
        print("timed out, as expected")

part 2:
    max_attempts = 5
    for attempt in range(max_attempts):
        try:
            response = requests.get(url, timeout=(3, 5))
            break # success – stop retrying
        except requests.exceptions.Timeout:
            if attempt == max_attempts - 1:
                raise # out of attempts, let the error propagate
            time.sleep(2 ** attempt)
```

```
# retry_backoff_practice.py – Intermediate section
import time
import requests

MAX_ATTEMPTS = 5

for attempt in range(MAX_ATTEMPTS):
    try:
        response = requests.get("http://10.255.255.1", timeout=(1, 2))
        break
    except requests.exceptions.Timeout:
        print(f"attempt {attempt} timed out")
        if attempt == MAX_ATTEMPTS - 1:
            ... # what goes here, once you're truly out of attempts?
        time.sleep(2 ** attempt)
```

The `if attempt == MAX_ATTEMPTS - 1:` check is the part people forget — without it, the loop just quietly finishes without telling you it never succeeded. Fill in that branch yourself, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
function compute_backoff_delay(attempt) -> float:
    base = 2 ** attempt
    jitter = a random float between 0 and 1
    return base + jitter

function get_with_retry(url, max_attempts=5) -> Response:
    for attempt in range(max_attempts):
        try:
            return requests.get(url, timeout=(3, 5))
        except requests.exceptions.Timeout as e:
            last_error = e
            if attempt < max_attempts - 1:
                sleep(compute_backoff_delay(attempt))
            raise TimeoutError(f"gave up after {max_attempts} attempts") from last_error
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# retry_backoff_practice.py – Intermediate section
import random
import time
import requests

def compute_backoff_delay(attempt: int) -> float:
    # your turn: 2 ** attempt, plus a small random amount of jitter
    ...

def get_with_retry(url: str, max_attempts: int = 5) -> requests.Response:
    last_error: Exception | None = None
    for attempt in range(max_attempts):
        try:
            return requests.get(url, timeout=(3, 5))
        except requests.exceptions.Timeout as e:
            last_error = e
            if attempt < max_attempts - 1:
                time.sleep(compute_backoff_delay(attempt))
            raise TimeoutError(f"gave up after {max_attempts} attempts") from last_error
```

Expected output, once filled in and called against a real URL that will eventually succeed on attempt 0: no timeout lines at all — `compute_backoff_delay` is never even called, since the first attempt returns immediately. You'll only see its effect by testing it directly:

`print(compute_backoff_delay(3))` should print something a little over `8.0` .

Fill in `compute_backoff_delay` yourself, then compare all 3 of your finished versions against the [Solution](#) — its Advanced version also explains exactly why this one function is worth pulling out on its own.

Difference between Basic, Intermediate, and Advanced: same loop shape at 3 completeness levels — Basic proves the pieces work individually. Intermediate combines them into one loop that correctly gives up and re-raises after the limit. Advanced adds jitter (so many failing callers don't retry in lockstep) and moves the backoff math into its own testable function, separate from the part of the code that actually sleeps and makes network calls — which is exactly the shape the Build Task's `compute_backoff_delay(attempt: int) -> float` needs to have.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)